

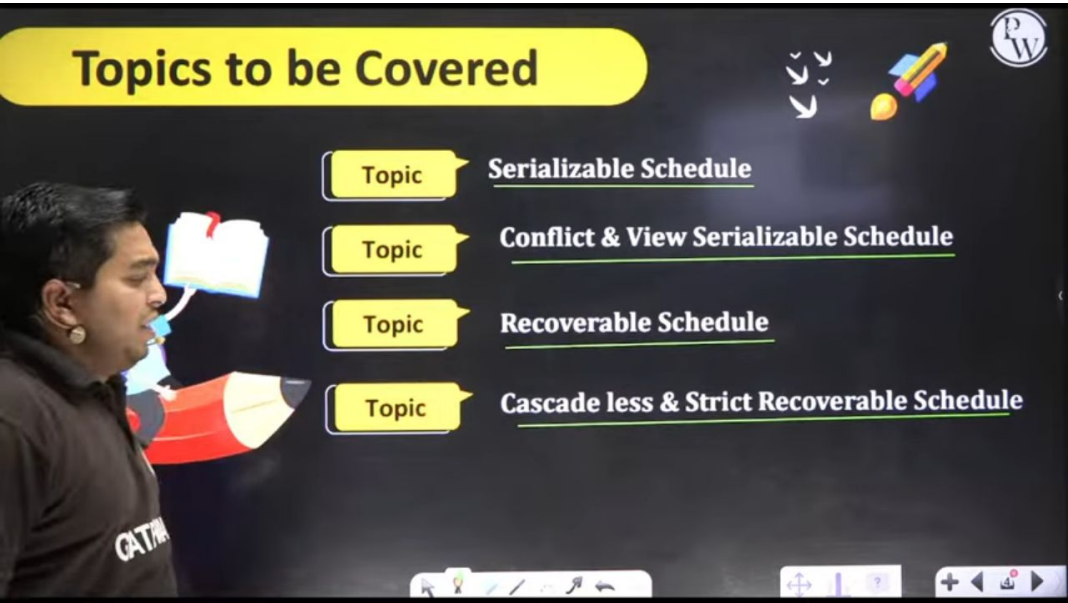


DBMS 06 | Serializable & Recoverable Schedule | CS & IT | GATE 2024 FastTrack Batch

Generated on October 23, 2024

Summary

AI Notes	AI Slides	Text Notes	Screenshots
✦ 0	✦ 0	📄 0	📷 64



▶ 2:32



Topic : Schedules



- ❑ Schedule: a sequences of instructions that specify the chronological order in which instructions of concurrent transactions are executed.
- ❖ A schedule for a set of transactions must consist of all instructions of the transactions
- ❖ Must preserve the order in which the instructions appear in each individual transaction.
- ❑ A transaction that successfully completes its execution will have a commit instructions as the last statement
- ❖ By default transaction assumed to execute commit instruction as its last step
- ❑ A transaction that fails to successfully complete its execution will have an abort instruction as the last statement.

▶ 4:23



Topic : Schedules



Let T_1 transfer 100 Rs from A to B, and T_2 transfer 10% of the balance from A to B.

Schedule 1

T_1	T_2
read (A) $A := A - 100$ write (A) read (B) $B := B + 100$ write (B) commit	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) Commit

$A: 1710$
 $B: 3290$
5000
Consistent
 $S_1 < T_1 T_2 >$

Schedule 2

T_1	T_2
read (A) $A := A - 100$ write (A) read (B) $B := B + 100$ write (B) commit	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) Commit

$A = 1700$
 $B = 3300$
5000
Consistent
 $S_2 < T_2 T_1 >$

▶ 5:15

Topic : Schedules

Non Serial [Concurrent execution]

Schedule 3

T ₁	T ₂
$A = 2000$ read (A) $A := A - 100$ write (A) $A = 1900$	read (A) $A = 1900$ $temp := A + 0.1$ $A := A - temp$ write (A) $A = 1710$
read (B) $B = 3000$ $B := B + 100$ write (B) $B = 3100$ commit $A: 1710$ $+ B: 3290$ $= 5000$ <u>Consistent</u>	read (B) $B = 3100$ $B := B + temp$ write (B) $B = 3290$ Commit

Schedule 4

T ₁	T ₂
$A = 2000$ read (A) $A := A - 100$ write (A) $A = 1900$	read (A) $A = 2000$ $temp := A + 0.1$ $A := A - temp$ write (A) $A = 1800$
read (B) $B = 3000$ $B := B + 100$ write (B) $B = 3100$ commit $A: 1900$ $+ B: 3300$ $= 5200$ <u>Inconsistent</u>	read (B) $B = 3100$ $B := B + temp$ write (B) $B = 3300$ Commit

▶ 18:04

Note Serial Schedule (All serial schedule (n!)) are always Consistent.

Note Non Serial schedule [Concurrent execution] may
 ⓐ May Not be Consistent.
 But we execute in Concurrent Manner.

▶ 18:39



Topic : Serial Schedule



- ❑ After Commit of one transaction, begins (Start) another transaction.
- ❑ Number of possible serial Schedules with 'n' transactions is "n!"
- ❑ The execution sequence of Serial Schedule always generates consistent result.

Example

S : $R_1(A)$ $W_1(A)$ Commit (T_1) $R_2(A)$ $W_2(A)$ commit (T_2).

▶ 19:13



Topic : Serial Schedule



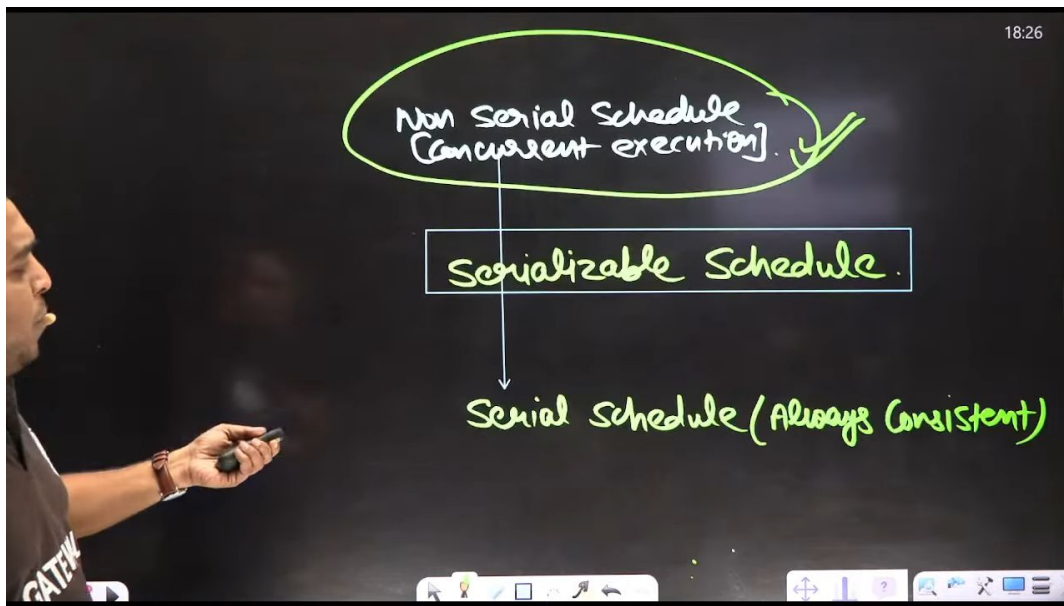
Advantage

- Serial Schedule always produce correct result (integrity guaranteed) as no resource sharing.

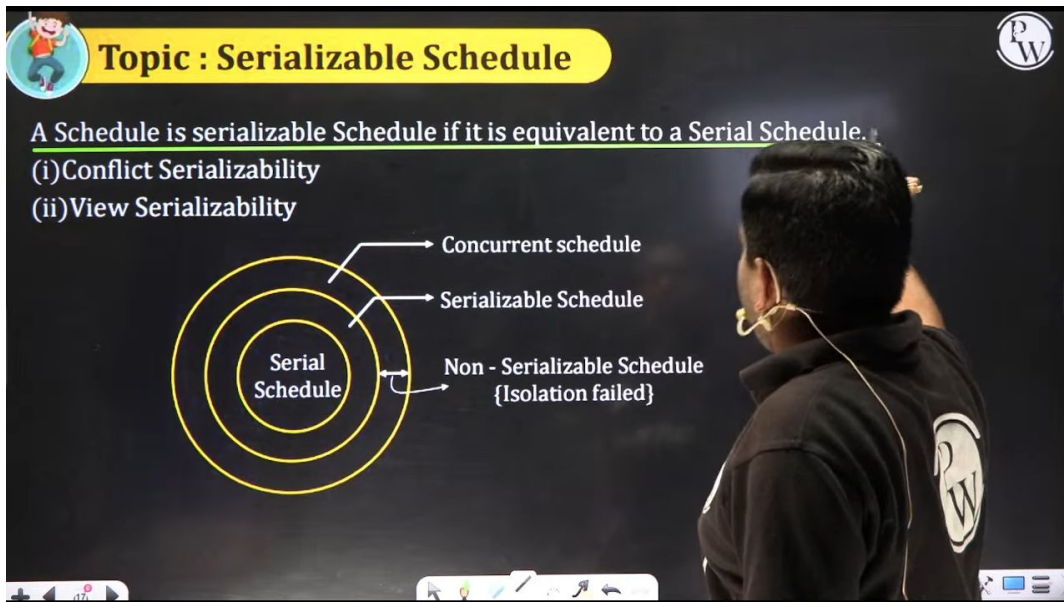
Disadvantage

- Less degree of concurrency.
- Through put of system is low.
- It allows transactions to execute one after another.

▶ 19:14



▶ 25:53



▶ 26:05

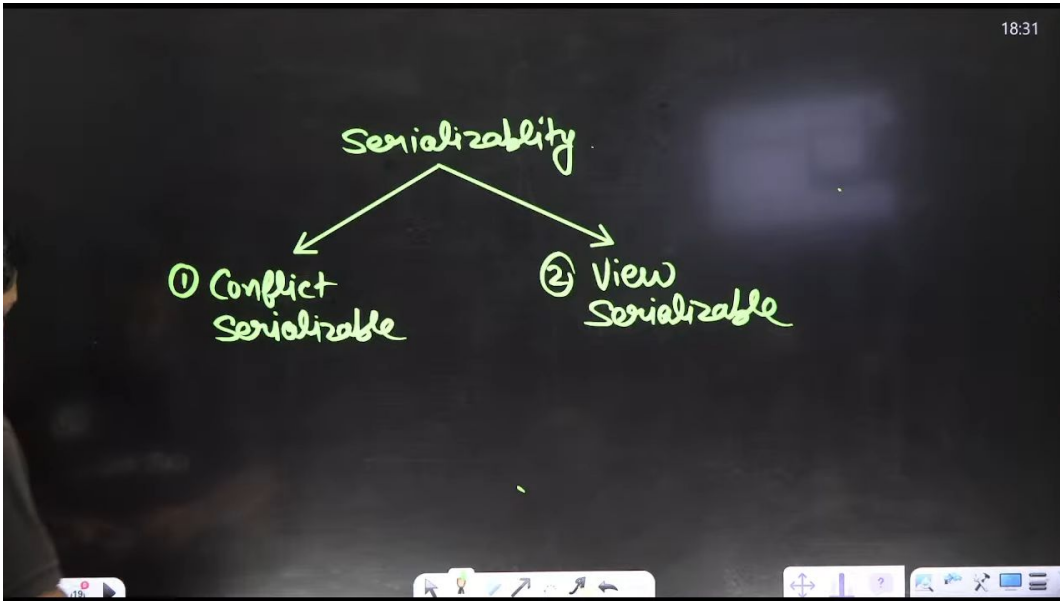


Topic : Serializability



- ❑ **Basic Assumption:** Each transaction preserves database consistency.
- ❑ Thus, serial execution of a set of transactions preserves database consistency.
- ❑ A (possibly concurrent) schedule is serializable if it is equivalent to a serial schedule. Different forms of schedule equivalence give rise to the notions of:
 1. Conflict serializability
 2. view serializability

▶ 27:39



▶ 30:12

① Conflict Serializable.

① Basic Concept Method

V. Imp.

② Testing (Precedence Graph) Method

③ Conflict Equivalent to Any serial Schedule.

▶ 33:20

Conflict serializable

Conflict Instruction

<u>T_i</u>	<u>T_j</u>	[Same Data Item]
$R(A)$	$W(A)$	} Conflict Instruction [Swapping Not allowed]
$W(A)$	$R(A)$	
$W(A)$	$W(A)$	
$R(A)$ operating on Different Data Item	$R(A)$	} Non Conflict Instruction (swapping allowed)

▶ 36:44



Topic : Conflict Serializability (Cont.)



- Schedule 3 can be transformed into Schedule 6, a serial schedule where T_2 follows T_1 , by series of swaps of non-conflicting instructions. Therefore Schedule 3 is conflict serializable.

Q

Schedule 3

T_1	T_2
read (A)	
Write (A)	
	read (A)
	write (A)
read (B)	
write (B)	
	read (B)
	write (B)

Schedule 6

T_1	T_2
read (A)	
write (A)	
read (B)	
write (B)	
	read (A)
	write (A)
	read (B)
	write (B)

▶ 43:01

!

Basic Concept method

Non Serial
Schedule

Swapping of
Non Conflicting
Instruction

Converted
into

Any Serial
Schedule

Schedule is
then Conflict Serializable

18:47

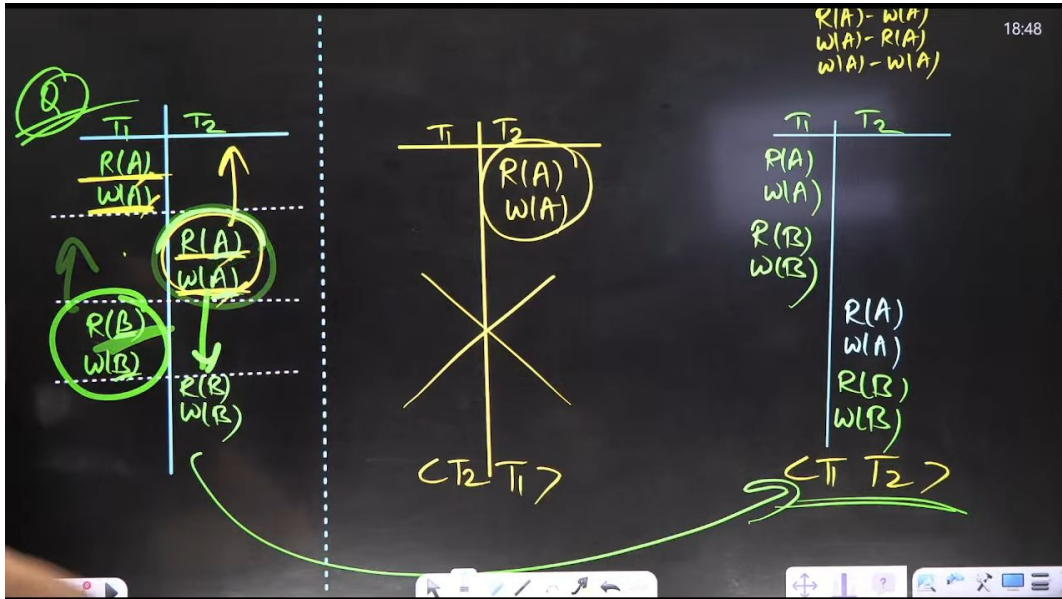
▶ 46:25

2 Serial Schedule.

$\langle T_1, T_2 \rangle$ T_1 followed by T_2 .

$\langle T_2, T_1 \rangle$ T_2 followed by T_1 .

▷ 47:01



▷ 47:40



Topic : Conflict Serializability (Cont.)

Example of a schedule that is not conflict serializable:

T ₃	T ₄
read (Q)	
	write (Q)
write (Q)	

We are unable to swap instructions in the above schedule to obtain either the serial schedule $\langle T_3, T_4 \rangle$, or the serial schedule $\langle T_4, T_3 \rangle$



48:48



Topic : Conflict Serializable

A schedule is said to be conflict serializable if it is conflict equivalent to a serial schedule.

Same conflicting operation order in C_1 & S_1

$\therefore$ Its $\{C_1\}$ conflict is conflict serializable.

T ₁	T ₂	T ₁	T ₂
read(A)		read(A)	
write(A)		write(A)	
	read(A)		read(A)
	write(A)		write(A)
			read(B)
read(B)			write(B)
write(B)			
	read(B)		read(B)
	write(B)		write(B)

C_1 S_1



51:07



Topic : Conflicting Instructions



- Instructions I_i and I_j of transactions T_i and T_j respectively, conflict if and only if there exists some item Q accessed by both I_i and I_j and at least one of these instructions wrote Q .
 1. $I_i = \text{read}(Q)$, $I_j = \text{read}(Q)$. I_i and I_j don't conflict.
 2. $I_i = \text{read}(Q)$ $I_j = \text{write}(Q)$. They conflict.
 3. $I_i = \text{write}(Q)$ $I_j = \text{read}(Q)$. They conflict.
 4. $I_i = \text{write}(Q)$ $I_j = \text{write}(Q)$. They conflict.
- Intuitively, a conflict between I_i and I_j forces a (logical) temporal order between them.
 - ❖ If I_i and I_j are consecutive in a schedule and they do not conflict, their results would remain the same even if they had been interchanged in the schedule.

▶ 51:11

② Testing for Conflict serializability

18:55

Precedence Graph Method

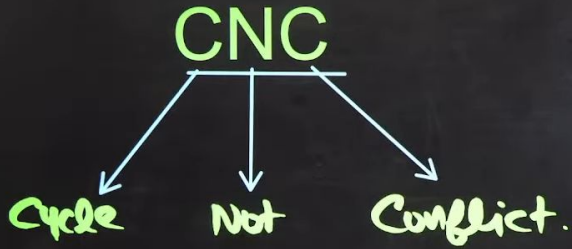
Directed Graph

$G : (V, E)$

V : Set of Transaction

E : Directed Edge Represent Conflict operation

▶ 54:21



Note

If Precedence Graph Contain Cycle [Any one Cycle]
then Schedule is Not Conflict Serializable.

▶ 57:40



Topic : Testing for Serializability



□ Testing for conflict serializability.

- ❖ Consider some schedule of a set of transactions $T_1, T_2, \dots, T_n$
- ❖ **Precedence graph** — a directed graph where the vertices are the transactions (names).
- ❖ We draw an arc from T_i to T_j if the two transaction conflict, and T_i accessed the data item on which the conflict arose earlier.
- ❖ We may label the arc by the item that was accessed.

▶ 57:57



Topic : Testing for Serializability

Example:



A schedule is conflict serializable if and only if its precedence graph is acyclic.

NOTE: CNC [Cycle not conflict serializable]

58:20



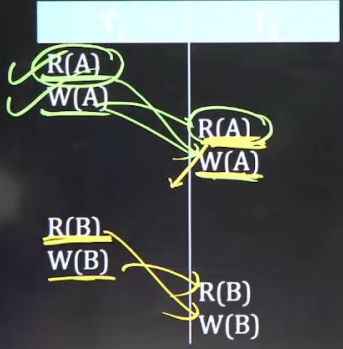
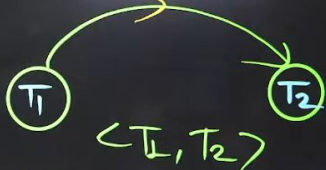
Topic : Testing for Serializability

[NAT]

#Q. S: $R_1(A) W_1(A) R_2(A) W_2(A) R_1(B) W_1(B) R_2(B) W_2(B)$

Same Data Item

R-W
W-R
W-W

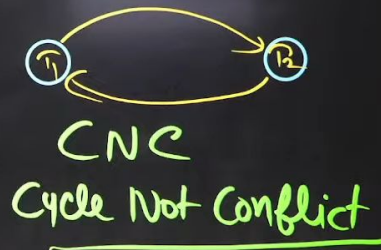
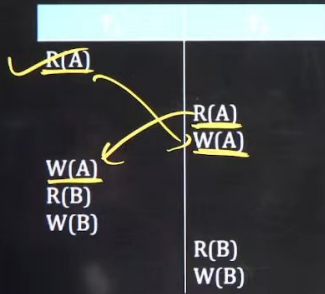
$\langle T_1, T_2 \rangle$
Conflict Serializable.

1:01:56

[NAT]



#Q. $R_1(A) R_2(A) W_2(A) W_1(A) R_1(B) W_1(B) R_2(B) W_2(B)$



inconsistent

▶ 1:11:01

[MCQ]

[2 Marks]



#Q. Consider the following four schedules due to three transactions (indicated by the subscript) using read and write on a data item x , denoted by $r(x)$ and $w(x)$ respectively. Which one of them is conflict serializable?

A $r_1(x); r_2(x); w_1(x); r_3(x); w_2(x)$

B $r_2(x); r_1(x); w_2(x); r_3(x); w_1(x)$

C $r_3(x); r_2(x); r_1(x); w_2(x); w_1(x)$

D $w_2(x); w_2(x); r_3(x); r_1(x); w_1(x)$

▶ 1:17:31

[MCQ]

[2 Marks]



#Q. Let $R_i(z)$ and $W_i(z)$ denote read and write operations on a data element z by a transaction T_i , respectively. Consider the schedule S with four transactions.

$S: R_4(x), R_2(x), R_3(x), R_1(y), W_1(y), W_2(x), W_3(y), R_4(y)$

Which one of the following serial schedules is conflict equivalent to S ?

A $T_1 \rightarrow T_3 \rightarrow T_4 \rightarrow T_2$

B $T_4 \rightarrow T_3 \rightarrow T_2$

$T_4 \rightarrow T_1 \rightarrow T_3 \rightarrow T_2$

D $T_3 \rightarrow T_1 \rightarrow T_4 \rightarrow T_2$

▶ 1:17:34

[MCQ]

[2 Marks]



#Q. Let $R_i(z)$ and $W_i(z)$ denote read and write operations on a data element z by a transaction T_i , respectively. Consider the schedule S with four transactions.

$S: R_4(x), R_2(x), R_3(x), R_1(y), W_1(y), W_2(x), W_3(y), R_4(y)$

Which one of the following serial schedules is conflict equivalent to S ?

A $T_1 \rightarrow T_3 \rightarrow T_4 \rightarrow T_2$

B $T_1 \rightarrow T_4 \rightarrow T_3 \rightarrow T_2$

C $T_4 \rightarrow T_1 \rightarrow T_3 \rightarrow T_2$

$T_3 \rightarrow T_1 \rightarrow T_4 \rightarrow T_2$

▶ 1:17:34

[MCQ]

[1 Marks]



#Q. Which one of the following is NOT a part of the ACID properties of database transactions?

- A Atomicity
- B Consistency
- C Isolation
- D Deadlock-freedom

▶ 1:17:36

[MCQ]

[1 Marks]



#Q. Consider the following transaction involving two bank accounts x and y.

read(x); x: = x - 50; write (x); read (y); y: = y + 50; write (y)

The constraint that the sum of the accounts x and y should remain constant is that of

- A Atomicity
- B Consistency
- C Isolation
- D Durability

▶ 1:17:36

[MCQ]

[1 Marks]



#Q. Consider the following transaction involving two bank accounts x and y.
read(x); x: = x - 50; write (x); read (y); y: = y + 50; write (y)
The constraint that the sum of the accounts x and y should remain constant is that of

- A. Atomicity
- B. Consistency
- C. Isolation
- D. Durability

▶ 1:17:38

[MCQ]

[2 Marks]



#Q. Consider the following schedule for transactions T1, T2 and T3:
Which one of the schedules below is the correct serialization of the above?

- A. T1 → T3 → T2
- B. T2 → T1 → T3
- C. T2 → T3 → T1
- D. T3 → T1 → T2

T1	T3	T3
Read(X)		
	Read (Y)	
		Read (Y)
	Write (Y)	
Write (X)		
		Write (X)
	Read (X)	
	Write (X)	

▶ 1:17:39

[MCQ]

[2 Marks]



#Q. Suppose a database schedule S involves transaction $T_1, \dots, T_n$. Construct the precedence graph of S with vertices representing the transactions and edges representing the conflicts. If S is serializable, which one of the following orderings of the vertices of the precedence graph is guaranteed to yield a serial schedule?

A Topological order

B Depth-first order

C Breadth-first order

D Ascending order of transaction indices

▶ 1:17:39



Topic : Conflict Equivalent Schedule



[NAT]

#Q. Two schedules are said to be conflict equivalent, if all conflicting operations in both the schedules must be executed in the same order.

$S_1: R_1(x) W_1(x) R_2(y) W_2(y) R_1(y)$

$S_2: R_1(x) W_1(x) R_1(y) R_2(y) W_2(y)$

▶ 1:17:41

[MCQ]



#Q. Consider two transactions T_1 and T_2 , and four schedules S_1, S_2, S_3, S_4 of T_1 and T_2 as given below:
 $T_1: R_1[x] W_1[x] W_1[y];$
 $T_2: R_2[x] R_2[y] W_2[y];$
 $S_1: R_1[x] R_2[x] R_2[y] W_1[x] W_1[y] W_2[y];$
 $S_2: R_1[x] R_2[x] R_2[y] W_1[x] W_2[y] W_1[y];$
 $S_3: R_1[x] W_1[x] R_2[x] W_1[y] R_2[y] W_2[y];$
 $S_4: R_2[x] R_2[y] R_1[x] W_1[x] W_1[y] W_2[y];$
 Which of the above schedules are conflict serializable?

- A S_1 and S_2 B S_2 and S_3
 C S_3 only D S_4 only

▶ 1:17:43



Topic : Conflict Equivalent Schedule



[NAT]

#Q. Two schedule are said to be conflict equivalent, if all conflicting operations in both the schedules must be executed in the same order.

$S_1: R_1(x) W_1(x) R_2(y) W_2(y) R_1(y)$

$S_2: R_1(x) W_1(x) R_1(y) R_2(y) W_2(y)$

T_1	T_2
$R(x)$	
$W(x)$	
	$R(y)$
	$W(y)$
$R(y)$	

$W_2(y) - R_1(y)$

OR

$W_1(y) - R_1(y) : T_2 \rightarrow T_1$

(S_1)

T_1	T_2
$R(x)$	
$W(x)$	
$R(y)$	
	$R(y)$
	$W(y)$

$R_1(y) - W_2(y)$

OR

$R_1(y) - W_1(y) : T_1 \rightarrow T_2$

S_1 is Not Conflict Equivalent to S_2

▶ 1:21:24



Topic : Conflict Equivalent Schedule

[NAT]

#Q. Two schedule are said to be conflict equivalent, if all conflicting operations in both the schedules must be executed in the same order.

$S_1: R_1(x) W_1(x) R_2(y) W_2(y) R_1(y)$
 $S_2: R_1(x) W_1(x) R_1(y) R_2(y) W_2(y)$

T_1	T_2
$R(x)$	
$W(x)$	
	$R(y)$
	$W(y)$
$R(y)$	

$W_2(y) - R_1(y)$
 $\textcircled{\text{OR}}$
 $W(y) - R(y) : T_2 \rightarrow T_1$
 $\textcircled{S_1}$

T_1	T_2
$R(x)$	
$W(x)$	
$R(y)$	
	$R(y)$
	$W(y)$

$R_1(y) - W_2(y)$
 $\textcircled{\text{OR}}$
 $R(y) - W(y) : T_1 \rightarrow T_2$

S_1 is Not Conflict Equivalent to S_2

▶ 1:21:24



Topic : Conflict Equivalent Schedule

[NAT]

#Q. Two schedule are said to be conflict equivalent, if all conflicting operations in both the schedules must be executed in the same order.

$S_1: R_1(x) W_1(x) R_2(y) W_2(y) R_1(y)$
 $S_2: R_1(x) W_1(x) R_1(y) R_2(y) W_2(y)$

T_1	T_2
$R(x)$	
$W(x)$	
	$R(y)$
	$W(y)$
$R(y)$	

$W_2(y) - R_1(y)$
 $\textcircled{\text{OR}}$
 $W(y) - R(y) : T_2 \rightarrow T_1$
 $\textcircled{S_1}$

T_1	T_2
$R(x)$	
$W(x)$	
$R(y)$	
	$R(y)$
	$W(y)$

$R_1(y) - W_2(y)$
 $\textcircled{\text{OR}}$
 $R(y) - W(y) : T_1 \rightarrow T_2$

S_1 is Not Conflict Equivalent to S_2

▶ 1:21:25

[NAT]



#Q. $S_1: R_1(A) W_1(A) R_2(A) W_2(A) R_1(B) W_1(B)$
 $S_2: R_1(A) W_1(A) R_2(A) R_1(B) W_2(A) W_1(B)$

T_1	T_2		
$R(A)$		$R_1(A) - W_2(A)$	$R(A) - W_2(A)$
$W(A)$		$W_1(A) - R_2(A)$	$W(A) - R_2(A)$
	$R(A)$	$W_1(A) - W_2(A)$	$W_1(A) - W_2(A)$
	$W(A)$		
$R(B)$			
$W(B)$			

OR

$R(A) - W_1(A): T_1 \rightarrow T_2$
 $W_1(A) - R(A): T_1 \rightarrow T_2$
 $W(A) - W_1(A): T_1 \rightarrow T_2$

OR

$R(A) - W_1(A): T_1 \rightarrow T_2$
 $W_1(A) - R(A): T_1 \rightarrow T_2$
 $W(A) - W_1(A): T_1 \rightarrow T_2$

S_1 & S_2 are Conflict Equivalent

▶ 1:24:44

[MCQ]

[2 Marks]



#Q. Consider a schedule of transactions T_1 and T_2 :

T_1	RA				RC		WD		WD	Commit	
T_2		RB	WB		RD		WC				Commit

Here, RX stands for "Read(X)" and WX stands for "Write(X)". Which one of the following schedules is conflict equivalent to the above schedule?

- A
- | | | | | | | | | | | |
|-------|----|----|----|----|----|----|----|----|--------|--------|
| T_1 | | | | | RA | RC | WD | WB | Commit | |
| T_2 | RB | WB | RD | WC | | | | | | Commit |
- B
- | | | | | | | | | | | |
|-------|----|----|----|--|----|----|----|----|--------|--------|
| T_1 | | | | | RA | RC | WD | WB | Commit | |
| T_2 | RB | WB | RD | | | | | WC | | Commit |
- C
- | | | | | | | | | | | |
|-------|----|----|----|----|----|----|----|----|--------|--------|
| T_1 | RA | RC | WD | WB | | | | | Commit | |
| T_2 | | | | | RB | WB | RD | WC | | Commit |
- D
- | | | | | | | | | | | |
|-------|----|----|----|----|----|----|----|----|--------|--------|
| T_1 | RA | RC | WD | WB | | | | | Commit | |
| T_2 | | | | | RB | WB | RD | WC | | Commit |

▶ 1:24:48

A S1, S2 and S3 are all conflict equivalent to each other

B No two of S1, S2 and S3 are conflict equivalent to each other

C S2 is conflict equivalent to S3, but not to S1

D S1 is conflict equivalent to S2, but not to S3

S1, S2 and S3 are all conflict equivalent to each other

No two of S1, S2 and S3 are conflict equivalent to each other

S2 is conflict equivalent to S 3, but not to S1

S1 is conflict equivalent to S2, but not to S3

Same conflicting operation order in C_1 & S_1

T_1	T_2	T_1	T_2
read(A)		read(A)	
write(A)		write(A)	
	read(A)	read(B)	
	write(A)	write(B)	
read(B)			read(A)
write(B)			write(A)
	read(B)		read(B)
	write(B)		write(B)
C_L		S_L	





Topic : View Serializability



Let S and S' be two schedules with the same set of transactions. S and S' are **view equivalent** if the following three conditions are met, for each data item Q ,

1. If in schedule S , transaction T_i reads the initial value of Q , then in schedule S' also transaction T_i must read the initial value of Q .
2. If in schedule S transaction T_i executes **read**(Q), and that value was produced by transaction T_j (if any), then in schedule S' also transaction T_i must read the value of Q that was produced by the same **write**(Q) operation of transaction T_j .
3. The transaction (if any) that performs the final **write**(Q) operation in schedule S must also perform the final **write**(Q) operation in schedule S' .

▶ 1:27:56

View serializable :

Same Data Item

- ① Initial Read
 - ② final write
 - ③ updated Read
(write - Read Sequence)
- Same

▶ 1:36:01

Note

If a Schedule is Conflict Serializable (Acyclic Precedence Graph) then it's already View Serializable.

Note

If a Schedule is Not Conflict Serializable then it May or May Not View Serializable.

▶ 1:37:01

19:45

①

S

A: Initial Read T_2
B: " " T_1

Initial Read

S'

A: Initial Read T_2
B: " " T_1

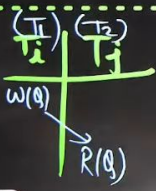
②

A: Final Write T_3
B: " " T_2

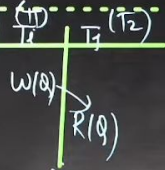
Final Write

A: Final Write T_3
B: " " T_2

③



Write-Read Sequence (Updated-Read)



▶ 1:44:25



Topic : View Serializability



- View Serializable Schedule: View equivalent serial schedule.
- View Equivalent: S_1 and S_2 said to be view equivalent.

Only if

- initial reads of S_1 and S_2 should be same.

Example

	T ₁	T ₂	T ₃
S ₁	R(A) (4-500) R(B) (4-500)	W(B) (3-200)	W(B)
S ₂	R(A) (3-200) R(B) (3-200)	W(B) (2-100)	W(B)

S_2
A: Initial Read T₁
B: Not Initial Read

$S_1 \neq S_2$

▶ 1:49:28



Topic : View Serializability



- Final updations for every data item should be same in S_1 and S_2

	T ₁	T ₂	T ₃
S ₁	W(A) W(B)	W(A)	W(A) W(B)
S ₂	W(A)	W(B) W(A)	W(B)

Final Write on A: T₃
B: T₃

A: T₂ (Not T₃)

$S_1 \neq S_2$

▶ 1:51:41



Topic : View Serializability (Cont.)



- A schedule S is **view serializable** if it is view equivalent to a serial schedule.
- Every conflict serializable schedule is also view serializable.
- Below is a schedule which is view-serializable but not conflict serializable.

Note:

Every view serializable schedule that is not conflict serializable has **blind writes**.

T		
read(Q)		
	write(Q)	
write(Q)		write(Q)

▶ 1:56:45

20:02

T_1 T_2 T_3

$w(Q)$

T_3
CNC

Check for View

$\langle T_1, T_2, T_3 \rangle$
 $\times \langle T_1, T_3, T_2 \rangle$
 $\times \langle T_2, T_1, T_3 \rangle$
 $\times \langle T_2, T_3, T_1 \rangle$
 $\times \langle T_3, T_1, T_2 \rangle$
 $\times \langle T_3, T_2, T_1 \rangle$

Dummy

① Initial Read
② Final Write
③ Write-Read

▶ 2:01:35

$\langle T_1, T_2, T_3 \rangle$

20:05

T_1	T_2	T_3
$r(Q)$		
$w(Q)$	$w(Q)$	
		$w(Q)$

Initial Read on Q: T_2
final write on Q: T_3
No WR Case

Not Conflict
But View Serializable
Serializable
(Blind Write)

T_1	T_2	T_3
$r(Q)$		
$w(Q)$	$w(Q)$	
		$w(Q)$

Initial Read on Q: T_1
Final Write on Q: T_3
No WR Case

2:04:17

[NAT]

#Q.

T_1	T_2	T_3
	$R(A)$ $R(B)$	
$W(B)$		
		$R(B)$
$W(A)$		
	$W(A)$	
		$W(A)$

2:04:18

[MCQ]

[1 Marks]



#Q. Consider the following transactions with data items P and Q initialized to zero:

T1: read (P);
read (Q);
if P = 0 then Q := Q + 1 ;
write (Q).

T2: read (Q) ;
read (P);
if Q = 0 then P := P + 1 ;
write (P).

Serial interleaving of T_1 and T_2 for concurrent execution leads to

▶ 2:04:19

a serializable schedule

a schedule that is not conflict serializable

a conflict serializable schedule

a schedule for which a precedence graph cannot be drawn

▶ 2:04:20

[MCQ]

[1 Marks]



#Q. Consider the following transactions with data items P and Q initialized to zero:

T1: read (P);
read (Q);
if P = 0 then Q := Q + 1 ;
write (Q).
T2: read (Q) ;
read (P);
if Q = 0 then P := P + 1 ;
write (P).

Any non-serial interleaving of T₁ and T₂ for concurrent execution leads to

▶ 2:04:20

[MCQ]

[2 Marks]



#Q. Consider the following schedule S of transactions T₁ and T₂:

Which of the following is TRUE about the schedule S?

S is serializable only as T₁, T₂

S is serializable only as T₂, T₁

C S is serializable both as T₁, T₂ and T₂ T₁

S is not serializable either as T₁ or as T₂

T ₁	T ₂
Read(A) A = A - 10	Read(A) Temp = 0.2 * A Write(A) Read(B)
Write(A) Read(B) B = B + 10 Write(B)	B = B + Temp Write(B)

▶ 2:04:21

[MCQ]

[2 Marks]



#Q. Consider a simple checkpointing protocol and the following set of operations in the log.
 (start, T4); (write, T4, y, 2, 3); (start, T1);
 (commit, T4); (write, T1, z, 5, 7);
 (checkpoint);
 (start, T2); (write, T2, x, 1, 9); (commit, T2);
 (start, T3); (write, T3, z, 7, 2);
 If a crash happens now and the system tries to recover using both undo and redo operations, what are the contents of the undo list and the redo list?

- A Undo: T1; Redo: T2 B Undo: T3, T1; Redo: T2, T4
 C Undo: none; Redo: T2, T4, T3, T1 D Undo: T3, T1, T4; Redo: T2

▶ 2:04:21

Topic : Finding Total Number of concurrent Schedule



T ₁	T ₂
R ₁ (A)	
W ₁ (A)	
	R ₂ (B)
	W ₂ (B)

T ₁	T ₂
L ₁	L ₃
L ₂	L ₄

T ₁	T ₂
0	1
0	1

L₁L₂L₃L₄
 L₃L₄L₁L₂
 L₁L₃L₂L₄ (or) L₁L₃L₄L₂
 L₃L₁L₄L₂ (or) L₃L₁L₂L₄

T₂ > S₂ < T₂ T₁ >
 (2)

T ₁	T ₂
R(A)	R(B)
W(A)	W(B)

T ₁	T ₂
R(A)	R(B)
W(A)	W(B)

T ₁	T ₂
R(A)	R(B)
W(A)	W(B)

T ₁	T ₂
R(A)	R(B)
W(A)	W(B)

T ₁	T ₂
R(A)	R(B)
W(A)	W(B)

(5)

▶ 2:04:22

Topic : Finding Total Number of concurrent Schedule

Total # Concurrent Schedule = $\frac{(n_1+n_2)!}{(n_1)!(n_2)!}$

$= \frac{(2+2)!}{(2)!(2)!} = \frac{4 \times 3 \times 2}{2 \times 2} = 6$

Total Concurrent = 6

Total non serial Schedule = Total Concurrent - Serial schedule($m!$)

m : # of transaction

$= 6 - 2$

Serial = 2

Non Serial = 4

$T_1 \rightarrow n_1$ operation
2 operation

$T_2 \rightarrow n_2$ operation
2 operation

▶ 2:04:22

Topic : Finding Total Number of concurrent Schedule

NOTE:

The Number of Concurrent schedule that can be formed for m transaction having $n_1, n_2, n_3, \dots, n_m$ operation respectively

Total # of Concurrent Schedule = $\frac{(n_1+n_2+n_3+\dots+n_m)!}{(n_1)!(n_2)!(n_3)! \dots (n_m)!}$

Total # of Non Serial Schedule = $\frac{(n_1+n_2+n_3+\dots+n_m)!}{(n_1)!(n_2)!(n_3)! \dots (n_m)!} - m!$

▶ 2:04:23



Topic : Finding Total Number of concurrent Schedule



$$\text{Total \# Concurrent Schedule} = \frac{(n_1 + n_2)!}{(n_1)!(n_2)!}$$

$$= \frac{(2+2)!}{(2)!(2)!} = \frac{4 \times 3 \times 2}{2 \times 2} = 6$$

$T_1 \rightarrow n_1 \text{ operation}$
 $\quad \quad 2 \text{ operation}$

$T_2 \rightarrow n_2 \text{ operation}$
 $\quad \quad 2 \text{ operation}$

$\text{Total Concurrent} = 6$

$\text{Total non serial Schedule} = \text{Total Concurrent} - \text{Serial schedule}(m!)$
 $\quad \quad \quad \quad \quad \quad \quad \quad \quad \quad \quad \quad m: \# \text{ of transaction}$

$= 6 - 2$
 $= 4$

$\text{Total non Serial} = 4$

▶ 2:04:24



Topic : Recoverable Schedules



Need to address the effect of transaction failures on concurrently running transactions.

■ **Recoverable schedule** — if a transaction T_j reads a data item previously written by a transaction T_i , then the commit operation of T_i appears before the commit operation of T_j .

The following schedule is not recoverable

T_8	T_9
read(A)	
write(A)	Read(A)
read(B)	commit

▶ 2:04:29



2 mins Summary

✓
Topic

Serializable Schedule

✓
Topic

Conflict & View Serializable Schedule

Topic

Recoverable Schedule

Topic

Cascade less & Strict Recoverable Schedule

▶ 2:04:45